

Chapitre 4

Analyse Sémantique et Typage

Théorie des Langages & Compilation

Dr Tegawendé F. BISSYANDÉ

Master Informatique – UFR/SEA
Université Joseph Ki-Zerbo



Plan du chapitre

- 1 Introduction à l'analyse sémantique
- 2 Surcharge
- 3 Vérification et synthèse de types
- 4 Inférence de types et unification
- 5 Polymorphisme
- 6 Analyse de portée
- 7 Systèmes de types
- 8 Exercices

Objectifs

- Vérifier que le programme a un **sens** au-delà de sa structure syntaxique
- Détecter les **erreurs sémantiques** (types incompatibles, variables non déclarées, etc.)
- Produire un arbre de syntaxe abstraite **annoté** (avec des informations de type)
- Préparer la **génération de code** efficace

Qualité d'un langage de programmation de haut niveau :

- Représenter **abstraitement** des opérations/données complexes
- **Réutiliser** le code dans des situations multiples
- Favoriser la **compréhension** du lecteur

Rappels sur les types

- Les **types** sont des objets structurés qui représentent les ensembles de valeurs possibles pour les résultats des programmes.
- Les types font en général partie du **langage de programmation**.
- On peut utiliser dans le compilateur des structures de types plus complexes (par exemple des **ensembles de types**) pour représenter des propriétés du programme.

Types de base : int, float, bool, char, string, unit

Types composés : tableaux, produits, listes, types fonctionnels

Types définis par l'utilisateur : enregistrements, types algébriques, classes

Arbre de syntaxe abstraite annoté

Les types peuvent être ajoutés en **décoration** de l'arbre de syntaxe abstraite :

```
type typ = ...
type profil = ...
type ('a, 'b) annot = { annot : 'a ; asa : 'b }
type funcs = (profil, ident) annot
type expr = (typ, expr_raw) annot
and expr_raw =
  | Cst of cte
  | Binop of op * expr * expr
  | If of expr * expr * expr
  | Call of funcs * expr list
```

Le typage prend en argument une expression avec des localisations et produit une expression **décorée par des types**.

Définition

La **surcharge** permet d'utiliser le même nom pour des constructions différentes :

- Opérations arithmétiques (entiers, flottants)
- Fonctions utilisateurs (Java, Ada)

La portée ne peut être résolue de manière syntaxique.

Approche sémantique : le contexte d'utilisation d'une expression devra permettre de déterminer **sans ambiguïté** l'opération concernée.

Résolution statique : on détermine à la compilation quelle fonction machine appeler.

Résolution dynamique : choix à l'exécution (Java : descripteur de classe ; Python : étiquette de type).

- Certaines opérations ont **plusieurs profils** possibles correspondant à des codes machines différents.
- La surcharge doit être complètement résolue **statiquement**.

Algorithme en deux passes :

- 1 **Phase ascendante** : des feuilles vers la racine, collecter l'ensemble des types possibles.
- 2 **Phase descendante** : de la racine vers les feuilles, propager le type choisi pour résoudre les ambiguïtés.

Exemple : résoudre $(1 + 4) + 3.2$ avec $+: (\text{int} \times \text{int} \rightarrow \text{int})$ et $+: (\text{float} \times \text{float} \rightarrow \text{float})$.

- Les types sont organisés par une notion d'**héritage simple** : $A \leq B$ si A est sous-classe de B
- On ne regarde que le type des **arguments** (et le sous-typage)
- Pour un appel $e.m(e_1, \dots, e_n)$, on cherche parmi les méthodes compatibles celle avec le profil **le plus spécifique**

Type statique vs. type dynamique :

- **Type statique** : déterminé à la compilation (approximation)
- **Type dynamique** : type réel à l'exécution
- Si e a pour type statique A , alors toute exécution de e donne un objet de type $B \leq A$

Vérification vs. synthèse de types

Environnement

Associe à chaque variable (symbole de fonction) son type (profil).

Vérification

- Les variables sont déclarées avec leur type
- On vérifie qu'une expression a un certain type
- Langages : C, Java, Rust

Synthèse (inférence)

- Pas de déclarations de types
- On déduit le type des variables et expressions
- Langages : OCaml, Haskell, ML

Types fonctionnels : $\tau_1 \rightarrow \tau_2$ (associativité à droite).

Règles de typage :

$$\text{Constante} \quad \frac{\text{type_cte}(c) = \tau}{\rho \vdash \text{Cst}(c) : \tau}$$

$$\text{Variable} \quad \frac{(x : \tau) \in \rho}{\rho \vdash \text{Var}(x) : \tau}$$

$$\text{Application} \quad \frac{\rho \vdash f : \tau' \rightarrow \tau \quad \rho \vdash e : \tau'}{\rho \vdash \text{App}(f, e) : \tau}$$

$$\text{Fonction} \quad \frac{\rho + (x : \tau') \vdash e : \tau}{\rho \vdash \text{Fun}(x, e) : \tau' \rightarrow \tau}$$

$$\text{Let} \quad \frac{\rho \vdash e_1 : \tau_1 \quad \rho + (x : \tau_1) \vdash e_2 : \tau_2}{\rho \vdash \text{Letin}(x, e_1, e_2) : \tau_2}$$

Problème

Comment deviner τ_i et τ quand on n'a pas de déclarations de types ?

$$\frac{\rho + (x_i : \tau_i)_{i=1..n} + (f : [\tau_1; \dots; \tau_n], \tau) \vdash e : \tau}{\rho \vdash \text{Fun}(f, [x_1; \dots; x_n], e) \text{ ok}}$$

Idée : utiliser les **contraintes d'égalité** entre types.

Technique : introduire des **variables de type** $\alpha, \beta, \dots$ et les résoudre par **unification**.

Algorithme de typage :

- 1 Pour `fun x -> e` : introduire une variable α pour le type de x , typer e , collecter les contraintes
- 2 Pour `f e` : vérifier que le type de f est de la forme $\tau_e \rightarrow \tau_r$ et unifier

Contraintes de type et unification

Les types ont une structure de **termes du premier ordre**.

Unification

Soient deux termes t et u avec des variables. On cherche s'il existe une **substitution** s telle que $s(t) = s(u)$.

Exemples d'unification :

Équation	Unificateur principal
int et α	$\{\alpha \mapsto \text{int}\}$
int et bool	Échec
α et β	$\{\alpha \mapsto \beta\}$
α et β array	$\{\alpha \mapsto \beta \text{ array}\}$
α et α array	Échec (pas de solution finie)

Principe

On traite un ensemble d'équations entre termes et on renvoie la substitution résultat.

Règles :

- 1 $\text{Var}(x) = \text{Var}(y)$ avec $x = y \Rightarrow$ ignorer
- 2 $\text{Var}(x) = t$ avec x dans $t \Rightarrow$ échec (**test d'occurrence**)
- 3 $\text{Var}(x) = t \Rightarrow$ ajouter $\{x \mapsto t\}$ et substituer dans les équations restantes
- 4 $\text{Term}(f, [t_1, \dots]) = \text{Term}(g, [u_1, \dots])$ avec $f \neq g \Rightarrow$ échec
- 5 $\text{Term}(f, [t_1, \dots, t_n]) = \text{Term}(f, [u_1, \dots, u_n]) \Rightarrow$ unifier $\{t_i = u_i\}$

Propriété : si une solution existe, l'algorithme renvoie l'**unificateur le plus général** (principal).

Exemple de typage

Typier **fun** $n \rightarrow$ **fun** $f \rightarrow$ **fun** $x \rightarrow f (n f x)$

- ❶ $n : \alpha, f : \beta, x : \gamma$. On a $f : \beta_1 \rightarrow \beta_2 \Rightarrow \beta = \beta_1 \rightarrow \beta_2$
- ❷ On type $n f x : \beta_1 \Rightarrow \alpha = \alpha_2 \rightarrow (\alpha_1 \rightarrow \beta_1)$
- ❸ $f : \alpha_2 \Rightarrow \alpha_2 = \beta_1 \rightarrow \beta_2$
- ❹ $x : \alpha_1 \Rightarrow \gamma = \alpha_1$

Résultat :

$$n : (\beta_1 \rightarrow \beta_2) \rightarrow (\alpha_1 \rightarrow \beta_1), \quad f : \beta_1 \rightarrow \beta_2, \quad x : \alpha_1$$

$$\text{Type} : ((\beta_1 \rightarrow \beta_2) \rightarrow \alpha_1 \rightarrow \beta_1) \rightarrow (\beta_1 \rightarrow \beta_2) \rightarrow \alpha_1 \rightarrow \beta_2$$

Expression non typable : **fun** $x \rightarrow (x x)$ échoue car $\alpha = \beta_1 \rightarrow \beta_2$ et $\beta_1 \rightarrow \beta_2 \simeq \beta_1$ n'a pas de solution finie.

Définition

Certaines constantes et fonctions sont **polymorphes** : elles ont un type contenant des variables universellement quantifiées.

Exemples :

- Conditionnelle : $\forall \alpha. \text{bool} \rightarrow \alpha \rightarrow \alpha \rightarrow \alpha$
- Accès dans un tableau : $\forall \alpha. \alpha \text{ array} \rightarrow \text{int} \rightarrow \alpha$
- Liste vide : $\forall \alpha. \alpha \text{ list}$
- Identité : $\forall \alpha. \alpha \rightarrow \alpha$

À chaque utilisation, on **instancie** les variables par des variables fraîches.

Généralisation

Les variables de type non déterminées après le typage d'une expression sont **généralisées** (quantifiées universellement).

Schéma de type : expression de type dont certaines variables sont quantifiées en tête : $\forall \alpha. \alpha \rightarrow \alpha$.

Vocabulaire :

Type monomorphe type sans variable de type (`int -> bool`)

Type polymorphe schéma de type ($\forall \alpha. \alpha \rightarrow \alpha$)

Instancier remplacer les variables quantifiées par des types concrets

Généraliser quantifier universellement les variables libres d'un type

Quand généraliser ?

Au moment des déclarations `let` :

$$\frac{\rho \vdash e_1 : \tau_1 \quad \rho + (x : \forall(\alpha_i).\tau_1) \vdash e_2 : \tau_2}{\rho \vdash \text{Letin}(x, e_1, e_2) : \tau_2}$$

Restriction : seules les variables **non libres dans l'environnement** peuvent être généralisées.

Exemple :

```
let id = fun x -> x      (* id : 'a -> 'a *)
(id true, id 1)          (* bien type ! *)
```

Mais :

```
(fun f -> (f true, f 1)) (fun x -> x)  (* erreur ! *)
```

Règle pour les définitions récursives

$$\frac{\rho + (f : \tau \rightarrow \tau')(x : \tau) \vdash e_1 : \tau' \quad \rho + (f : \tau \rightarrow \tau') \vdash e_2 : \tau_2}{\rho \vdash \text{LetRec}(f, \text{fun } x \rightarrow e_1, e_2) : \tau_2}$$

On introduit deux variables α et β pour τ et τ' , puis on résout les contraintes.

Exemple :

```
let rec fact n =  
  if n <= 0 then 1  
  else n * fact (n - 1)  
(* fact : int -> int *)
```

Problème des références polymorphes

Sans restriction, le programme suivant serait bien typé mais provoquerait une erreur à l'exécution :

```
let lr = ref []  
let add x = lr := x :: !lr; !lr  
add 1; add 2; add true  (* erreur a l'execution ! *)
```

Solution (restriction de valeur) : le type d'une expression **expansive** (de la forme $e_1 \ e_2$) n'est **pas** généralisé.

Seules les **valeurs** (fonctions, constantes) peuvent avoir des types généralisés. Cela garantit : « tout programme bien typé ne fait pas d'erreur de type à l'exécution ».

Table des symboles et portée

Table des symboles

Structure de données qui associe chaque identificateur à ses **attributs** : type, adresse mémoire, portée, etc. Consultée et mise à jour par toutes les phases.

Portée (*scope*) : région du programme où un identificateur est visible.

Portée statique (lexicale)

Déterminée par la **structure du texte**. Utilisée par C, Java, OCaml, Python, ...

Portée dynamique

Déterminée par l'**ordre d'exécution**. Utilisée par certains Lisp, Bash, Emacs Lisp.

Implémentation : pile d'environnements (un par bloc/fonction) ou table de hachage avec chaînage.

Exemple de portée imbriquée

```
int x = 1;           // x_global
void f() {
    int x = 2;       // x_local a f
    {
        int x = 3;    // x_local au bloc
        printf("%d", x); // affiche 3
    }
    printf("%d", x);   // affiche 2
}
// ici x vaut 1 (x_global)
```

L'analyse de portée doit résoudre **chaque occurrence** d'un identificateur vers sa **déclaration** correcte.
Erreur si variable utilisée mais non déclarée.

Typage statique vs. dynamique

Typage statique

Vérification des types **à la compilation.**

- Erreurs détectées tôt
- Code plus efficace
- C, Java, OCaml, Rust, Haskell

Typage dynamique

Vérification des types **à l'exécution.**

- Plus flexible
- Erreurs tardives
- Python, Ruby, JavaScript, Lisp

Typage fort vs. faible

- **Fort** : peu ou pas de conversions implicites (OCaml, Rust, Haskell)
- **Faible** : conversions implicites fréquentes (C, JavaScript, PHP)

Types de base et types composés

Types de base

- **Numériques** : `int`, `float`, `double`, `long`
- **Booléen** : `bool` (`true/false`)
- **Caractère** : `char`
- **Unité** : `void` (C), `unit` (OCaml)

Constructeurs de types

- **Tableaux** : τ array – accès indexé en $O(1)$
- **Produits** (tuples, structures) : $\tau_1 \times \tau_2 \times \dots \times \tau_n$
- **Sommes** (unions étiquetées, variants) : $\tau_1 + \tau_2$
- **Fonctions** : $\tau_1 \rightarrow \tau_2$
- **Références** : τ ref – cases mémoire mutables
- **Types paramétrés** : α list, (α, β) map

Coercions et conversions de types

Conversion implicite (coercion)

Le compilateur insère automatiquement une opération de conversion quand les types ne correspondent pas exactement. Exemple en C : `int` $\rightarrow$ `float` dans `3 + 2.5`.

Règles de coercion :

- **Élargissement** (*widening*) : `int` $\rightarrow$ `long` $\rightarrow$ `float` $\rightarrow$ `double` (sans perte)
- **Rétrécissement** (*narrowing*) : `double` $\rightarrow$ `int` (perte potentielle, souvent avertissement)

Conversion explicite (cast)

Le programmeur demande explicitement la conversion : `(int) 3.14` en C/Java.

En **OCaml**, aucune coercion implicite : `3 + 2.5` est une erreur, il faut écrire `float_of_int 3 +. 2.5`.

Équivalence de types

Comment déterminer si deux types sont “les mêmes” ?

Équivalence structurelle

Deux types sont équivalents s'ils ont la **même structure**.

- Utilisée par C, OCaml, ML
- `type A = int * int`
- `type B = int * int`
- $A \equiv B$ (même structure)

Équivalence nominale

Deux types sont équivalents s'ils ont le **même nom**.

- Utilisée par Java, Ada, Rust
- `class A { int x, y; }`
- `class B { int x, y; }`
- $A \neq B$ (noms différents)

Définition

A est un **sous-type** de B (noté $A \leq B$) si toute valeur de type A peut être utilisée là où une valeur de type B est attendue.

Exemples :

- En Java : $\text{Integer} \leq \text{Number} \leq \text{Object}$
- Types numériques : $\text{int} \leq \text{float} \leq \text{double}$ (par coercion)

Sous-typage des fonctions (règle de **variance**) :

$$\frac{B_1 \leq A_1 \quad A_2 \leq B_2}{A_1 \rightarrow A_2 \leq B_1 \rightarrow B_2}$$

Le type des arguments est **contravariant** (inversé) et le type de retour est **covariant** (même sens). Ce principe est fondamental en programmation orientée objet.

Propriétés

L'algorithme W est l'algorithme canonique d'inférence de types pour les langages de la famille ML. Il possède les propriétés suivantes :

- **Complet** : si un type existe, il le trouve
- **Principal** : il trouve le type **le plus général**
- **Décidable** : il termine toujours
- **Complexité** : quasi-linéaire en pratique (exponentiel dans le pire cas théorique)

Les étapes :

- 1 Attribuer des **variables de type fraîches** aux expressions sans type connu
- 2 Parcourir l'AST et générer des **équations de types**
- 3 Résoudre par **unification** (trouver l'unificateur le plus général)
- 4 **Généraliser** les variables restantes dans les let

Principe

Associer des **actions sémantiques** (calcul d'attributs) aux règles de la grammaire. Les attributs sont propagés lors de la construction de l'arbre syntaxique.

Attributs synthétisés : calculés à partir des fils du nœud (bottom-up).

Attributs hérités : calculés à partir du parent et des frères (top-down).

Exemple : calculatrice d'expressions

Production	Action sémantique
$E ::= E_1 + T$	$E.val = E_1.val + T.val$
$E ::= T$	$E.val = T.val$
$T ::= T_1 * F$	$T.val = T_1.val \times F.val$
$F ::= \text{int}$	$F.val = \text{int.lexval}$

Définitions dirigées par la syntaxe (SDD) – Exemple complet

Problème : convertir un nombre binaire en décimal.

Grammaire : $B ::= B_1 d \mid d$ avec $d \in \{0, 1\}$

Attributs synthétisés :

Production	Règles sémantiques
$B ::= B_1 d$	$B.val = 2 \times B_1.val + d.val$
$B ::= d$	$B.val = d.val$
$d ::= 0$	$d.val = 0$
$d ::= 1$	$d.val = 1$

Exemple : $101 \rightarrow B.val = 2 \times (2 \times 1 + 0) + 1 = 5$

Vérification sémantique : au-delà du typage

Le typage n'est qu'une partie de l'analyse sémantique. D'autres vérifications sont nécessaires :

- **Vérification de portée** : toute variable utilisée est-elle déclarée ?
- **Unicité** : pas de déclaration en double dans le même bloc
- **Contrôle de flot** :
 - break et continue uniquement dans des boucles
 - return dans chaque chemin d'une fonction non-void
- **Vérification d'initialisation** : variable lue avant d'être écrite ?
- **Accessibilité** : membres private/protected en POO
- **Exhaustivité** : pattern matching complet (OCaml, Rust)
- **Analyse de durée de vie** : emprunt (*borrow checking*) en Rust

Spécificités

- **Héritage** : $A \leq B$ implique que tout objet A est aussi un B
- **Redéfinition de méthodes** (*override*) : la méthode appelée dépend du type **dynamique** de l'objet (liaison tardive / *late binding*)
- **Classes abstraites et interfaces** : définissent des contrats de type
- **Généricité** : `List<T>` en Java, templates en C++

Surcharge vs. redéfinition :

- **Surcharge** (*overloading*) : résolue **statiquement** selon le type des arguments
- **Redéfinition** (*overriding*) : résolue **dynamiquement** selon le type réel de l'objet (via la vtable / table des méthodes virtuelles)

Exercice 1 – Vérification de types

- ❶ Donner le type des expressions OCaml suivantes (en utilisant les règles de typage) :
 - `fun x -> x + 1`
 - `fun x -> fun y -> if x then y else y + 1`
 - `fun f -> fun x -> f (f x)`
- ❷ L'expression `fun x -> (x x)` est-elle typable ? Justifier en détaillant l'algorithme d'unification.
- ❸ Donner le type le plus général de :
 - `fun f -> fun g -> fun x -> f (g x)`
 - `fun x -> (fst x, snd x)`

Exercice 2 – Unification

Résoudre les équations de types suivantes (ou montrer qu'il n'y a pas de solution) :

- ❶ $\alpha \rightarrow \text{int} = \text{bool} \rightarrow \beta$
- ❷ $\alpha \rightarrow \alpha = \text{int} \rightarrow \text{bool}$
- ❸ $(\alpha \rightarrow \beta) \rightarrow \gamma = (\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})$
- ❹ $\alpha \text{ list} = \text{int list list}$
- ❺ $\alpha \rightarrow \beta = (\beta \rightarrow \gamma) \rightarrow \gamma$

Pour chaque équation soluble, donner l'**unificateur principal** (substitution la plus générale).

Exercice 3 – Inférence de types

Appliquer l'algorithme d'inférence de types (avec variables de type et unification) aux expressions suivantes :

- 1 `let compose f g x = f (g x) in compose`
- 2 `let rec map f = function [] -> [] | x::xs -> (f x) :: (map f xs)`
- 3 `let rec fold f acc = function [] -> acc | x::xs -> fold f (f acc x) xs`

Détailler les étapes : introduction des variables de type, équations générées, et résolution par unification.

Exercice 4 – Polymorphisme

- ❶ Expliquer la différence entre les deux programmes OCaml suivants :

```
(* Programme A *)
let id = fun x -> x in (id true, id 1)

(* Programme B *)
(fun f -> (f true, f 1)) (fun x -> x)
```

Pourquoi A est accepté et B est rejeté par le typeur ?

- ❷ Ordonner les types suivants du plus général au plus spécifique :
- $\alpha \rightarrow \alpha$
 - $(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$
 - $\text{int} \rightarrow \text{int}$
 - $\alpha \rightarrow \text{int}$
- ❸ Expliquer pourquoi `let r = ref []` donne le type `'a list ref` et non `'a list ref` en OCaml.